

Introduction to GitHub Copilot in Testing

This module introduces how **Generative AI and GitHub Copilot** are transforming software testing practices.

What is Software Testing?

Software Testing is the process of evaluating a software application to:

- Identify defects
- Ensure functionality works as expected
- Validate performance, security, and usability

Main Objectives

- Find bugs
 - Verify requirements
 - Improve quality
 - Reduce risk
-

Types of Software Testing

- Unit Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing
 - Performance Testing
 - Security Testing
-

Generative AI in Testing

Generative AI uses machine learning models to:

- Generate test cases
- Create test scripts
- Suggest edge cases
- Generate test data
- Automate regression testing

Traditional vs. Generative AI Testing

Aspect	Traditional Testing	Generative AI Testing
Test creation	Manual	Automatic
Speed	Slow	Fast
Coverage	Limited	Wide
Maintenance	High	Low
Scalability	Difficult	Easy

Applications of Generative AI in Testing

- Automated test case generation
 - Unit test creation
 - UI test scripts
 - API testing
 - Test data generation
 - Bug reproduction steps
-

Generative AI Testing: Benefits

- Faster testing cycles
 - Better test coverage
 - Reduced human effort
 - Improved reliability
 - Cost savings
-

How GitHub Copilot Helps in Testing

- Generates unit tests
 - Creates mock data
 - Suggests assertions
 - Writes test frameworks code
-

Demo: Comparing Traditional and Generative AI-based Test Case Creation

Scenario: Login Function

```
def login(username, password):  
    if username == "admin" and password == "1234":  
        return "Success"  
    return "Fail"
```

Traditional Test Case (Manual)

```
def test_login_success():  
    assert login("admin", "1234") == "Success"  
  
def test_login_fail():  
    assert login("user", "pass") == "Fail"
```

Generative AI-Based (Copilot Generated)

Add comment:

```
# Write pytest unit tests for login function
```

Copilot generates:

```
import pytest  
  
def test_login_valid_credentials():  
    assert login("admin", "1234") == "Success"  
  
def test_login_invalid_username():  
    assert login("user", "1234") == "Fail"  
  
def test_login_invalid_password():  
    assert login("admin", "wrong") == "Fail"  
  
def test_login_empty_inputs():  
    assert login("", "") == "Fail"
```

Observations

- AI generated more test cases
 - Covered edge cases
 - Faster creation
-

Best Practices

- Review generated tests
 - Add domain-specific scenarios
 - Run tests frequently
-

Limitations

- Might miss business rules
 - Needs human validation
-

Quick Check

Q1. What is software testing?

Process of finding defects

Q2. True/False

Generative AI can generate test cases automatically.
True

Q3. Which is faster?

- A) Manual testing
- B) Generative AI testing

B

Summary

GitHub Copilot enhances testing by:

- ✓ Generating tests
- ✓ Improving coverage
- ✓ Reducing effort

But **human testers remain essential.**